```
"""
Q1
Program: Significance of Classes
Author: Jasstin Roy
"""

class Example:
    def __init__(self, name):
        self.name = name

    def display(self):
        print("Classes help organize code using objects.")
        print("Name:", self.name)

obj = Example("Rahul")
obj.display()

print("Jasstin Roy")
```

```
Classes help organize code using objects.
Name: Rahul
Jasstin Roy
```

```
"""
Q2
Program: Bank Account Class
Author: Jasstin Roy
"""

class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print("Deposited:", amount)

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Withdrawn:", amount)
        else:
            print("Insufficient balance")

    def display(self):
        print("Balance:", self.balance)

acc = BankAccount(1000)
acc.deposit(500)
acc.withdraw(200)
acc.display()

print("Jasstin Roy")
```

```
Deposited: 500
Withdrawn: 200
Balance: 1300
Jasstin Roy
```

```
"""
Q3
Program: Book with Chapters
Author: Jasstin Roy
"""

class Chapter:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

class Book:
    def __init__(self, chapters):
        self.chapters = chapters
```

```
    def total_pages(self):
        total = sum(ch.pages for ch in self.chapters)
        print("Total pages:", total)

ch1 = Chapter("Intro", 10)
ch2 = Chapter("Middle", 20)
ch3 = Chapter("End", 15)

book = Book([ch1, ch2, ch3])
book.total_pages()

print("Jasstin Roy")
```

```
Total pages: 45
Jasstin Roy
```

```
"""
Q4
Program: Access Control
Author: Jasstin Roy
"""

class Demo:
    def __init__(self):
        self.public = "Public"
        self._protected = "Protected"
        self.__private = "Private"

    def show(self):
        print(self.public)
        print(self._protected)
        print(self.__private)

obj = Demo()
obj.show()

print("Jasstin Roy")
```

```
Public
Protected
Private
Jasstin Roy
```

```
"""
Q5
Program: Time Conversion
Author: Jasstin Roy
"""

class Time:
    def __init__(self, time_str):
        self.time_str = time_str

    def is_valid(self):
        try:
            h, m, s = map(int, self.time_str.split(":"))
            return 0 <= h < 24 and 0 <= m < 60 and 0 <= s < 60
        except:
            return False

    def convert(self):
        if not self.is_valid():
            print("Invalid time")
            return

        h, m, s = map(int, self.time_str.split(":"))
        period = "AM"

        if h >= 12:
            period = "PM"
            if h > 12:
                h -= 12
        elif h == 0:
            h = 12

        print(f"{h}:{m}:{s} {period}")
```

```
t = Time("14:30:20")
t.convert()

print("Jasstin Roy")
```

```
2:30:20 PM
Jasstin Roy
```

```
"""
Q6
Program: Private Bank Account
Author: Jasstin Roy
"""

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient funds")

    def display(self):
        print("Balance:", self.__balance)

acc = BankAccount(1000)
acc.deposit(500)
acc.withdraw(200)
acc.display()

print("Jasstin Roy")
```

```
Balance: 1300
Jasstin Roy
```

```
"""
Q7
Program: Card Game Simulation
Author: Jasstin Roy
"""

import random

class Card:
    def __init__(self, suit, rank):
        self.suit = suit
        self.rank = rank

    def __str__(self):
        return f"{self.rank} of {self.suit}"

class Deck:
    def __init__(self):
        suits = ["Hearts", "Spades", "Clubs", "Diamonds"]
        ranks = ["2","3","4","5","6","7","8","9","10","J","Q","K","A"]
        self.cards = [Card(s, r) for s in suits for r in ranks]

    def shuffle(self):
        random.shuffle(self.cards)

    def deal(self, n):
        return [self.cards.pop() for _ in range(n)]

class Player:
    def __init__(self, name):
        self.name = name
        self.hand = []

    def receive(self, cards):
        self.hand.extend(cards)
```

```
    def show(self):
        print(self.name, "has:")
        for c in self.hand:
            print(c)

deck = Deck()
deck.shuffle()

p1 = Player("Player1")
p2 = Player("Player2")

p1.receive(deck.deal(3))
p2.receive(deck.deal(3))

p1.show()
p2.show()

print("Jasstin Roy")
```

```
Player1 has:
5 of Diamonds
4 of Hearts
9 of Diamonds
Player2 has:
5 of Hearts
3 of Clubs
4 of Clubs
Jasstin Roy
```

```
"""
Q8
Program: Inheritance (Vehicle and Car)
Author: Jasstin Roy
"""

class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def display_info(self):
        print("Vehicle:", self.make, self.model)

class Car(Vehicle):
    def __init__(self, make, model, num_doors):
        super().__init__(make, model)
        self.num_doors = num_doors

    def display_info(self):
        super().display_info()
        print("Doors:", self.num_doors)

v = Vehicle("Toyota", "Corolla")
c = Car("Honda", "City", 4)

v.display_info()
c.display_info()

print("Jasstin Roy")
```

```
Vehicle: Toyota Corolla
Vehicle: Honda City
Doors: 4
Jasstin Roy
```

```
"""
Q9
Program: Polymorphism
Author: Jasstin Roy
"""

import math

class Shape:
    def area(self):
        pass
```

```
class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return math.pi * self.r * self.r

class Rectangle(Shape):
    def __init__(self, l, w):
        self.l = l
        self.w = w

    def area(self):
        return self.l * self.w

c = Circle(5)
r = Rectangle(4, 6)

print("Circle Area:", c.area())
print("Rectangle Area:", r.area())

print("Jasstin Roy")
```

```
Circle Area: 78.53981633974483
Rectangle Area: 24
Jasstin Roy
```

```
"""
Q10
Program: Commission Employee
Author: Jasstin Roy
"""

class CommissionEmployee:
    def __init__(self, name, sales, rate):
        self.name = name
        self.set_sales(sales)
        self.set_rate(rate)

    def set_sales(self, sales):
        if sales < 0:
            raise ValueError("Sales cannot be negative")
        self.sales = sales

    def set_rate(self, rate):
        if rate < 0:
            raise ValueError("Rate cannot be negative")
        self.rate = rate

    def earnings(self):
        return self.sales * self.rate

    def __repr__(self):
        return f"{self.name} earns {self.earnings()}"

emp = CommissionEmployee("John", 10000, 0.1)
print(emp)

emp.set_sales(20000)
print("Updated Earnings:", emp.earnings())

print("Jasstin Roy")
```

```
John earns 1000.0
Updated Earnings: 2000.0
Jasstin Roy
```

```
"""
Q11
Program: Duck Typing
Author: Jasstin Roy
"""

class Dog:
    def speak(self):
```

```
        print("Dog barks")

class Robot:
    def speak(self):
        print("Robot speaks")

def describe(obj):
    obj.speak()

d = Dog()
r = Robot()

describe(d)
describe(r)

print("Jasstin Roy")
```

```
Dog barks
Robot speaks
Jasstin Roy
```

```
"""
Q12
Program: Operator Overloading
Author: Jasstin Roy
"""

class Complex:
    def __init__(self, r, i):
        self.r = r
        self.i = i

    def __add__(self, other):
        return Complex(self.r + other.r, self.i + other.i)

    def __str__(self):
        return f"{self.r} + {self.i}i"

c1 = Complex(2, 3)
c2 = Complex(4, 5)

c3 = c1 + c2
print("Result:", c3)

print("Jasstin Roy")
```

```
Result: 6 + 8i
Jasstin Roy
```

```
"""
Q13
Program: Custom Exception
Author: Jasstin Roy
"""

class InsufficientFunds(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount

    def __str__(self):
        return f"Balance: {self.balance}, Withdrawal: {self.amount}"

class Bank:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFunds(self.balance, amount)
        self.balance -= amount

b = Bank(1000)

try:
    b.withdraw(1500)
```

```
except InsufficientFunds as e:
    print(e)

print("Jasstin Roy")
```

```
Balance: 1000, Withdrawal: 1500
Jasstin Roy
```

```
"""
Q14
Program: Card Dataclass Deal 5 Cards
Author: Jasstin Roy
"""

from dataclasses import dataclass
import random

@dataclass
class Card:
    suit: str
    rank: str

class Deck:
    def __init__(self):
        suits = ["Hearts","Spades","Clubs","Diamonds"]
        ranks = ["2","3","4","5","6","7","8","9","10","J","Q","K","A"]
        self.cards = [Card(s, r) for s in suits for r in ranks]

    def shuffle(self):
        random.shuffle(self.cards)

    def deal(self, n):
        return [self.cards.pop() for _ in range(n)]

deck = Deck()
deck.shuffle()

hand = deck.deal(5)

print("Player Hand:")
for c in hand:
    print(c)

print("Remaining cards:", len(deck.cards))
print("Jasstin Roy")
```

```
Player Hand:
Card(suit='Spades', rank='8')
Card(suit='Clubs', rank='10')
Card(suit='Clubs', rank='4')
Card(suit='Hearts', rank='5')
Card(suit='Spades', rank='K')
Remaining cards: 47
Jasstin Roy
```

```
"""
Q15
Program: Dataclass Advantage
Author: Jasstin Roy
"""

from dataclasses import dataclass

@dataclass
class Student:
    name: str
    age: int

s = Student("Alice", 20)

print(s)
print("Dataclasses are mutable and more flexible than namedtuples")

print("Jasstin Roy")
```

```
Student(name='Alice', age=20)
Dataclasses are mutable and more flexible than namedtuples
```

```
Jasstin Roy
```

```
"""
Q16
Program: Doctest Example
Author: Jasstin Roy

This function returns the sum of two numbers.

Test Cases:
>>> add(2, 3)
5
>>> add(-1, 1)
0
>>> add(0, 0)
0
>>> add(10, -5)
5
"""

def add(a, b):
    """
    Returns sum of a and b.

    >>> add(2, 3)
    5
    >>> add(5, 5)
    10
    """
    return a + b


if __name__ == "__main__":
    import doctest

    # Run all doctest cases
    doctest.testmod(verbose=True)

    print("All tests executed successfully")
    print("Jasstin Roy")
```

```
Trying:
    add(2, 3)
Expecting:
    5
ok
Trying:
    add(-1, 1)
Expecting:
    0
ok
Trying:
    add(0, 0)
Expecting:
    0
ok
Trying:
    add(10, -5)
Expecting:
    5
ok
Trying:
    add(2, 3)
Expecting:
    5
ok
Trying:
    add(5, 5)
Expecting:
    10
ok
74 items had no tests:
    __main__.Animal
    __main__.Animal.__init__
    __main__.Animal.display_species
    __main__.Bank
    __main__.Bank.__init__
    __main__.Bank.withdraw
    __main__.BankAccount
    __main__.BankAccount.__init__
```

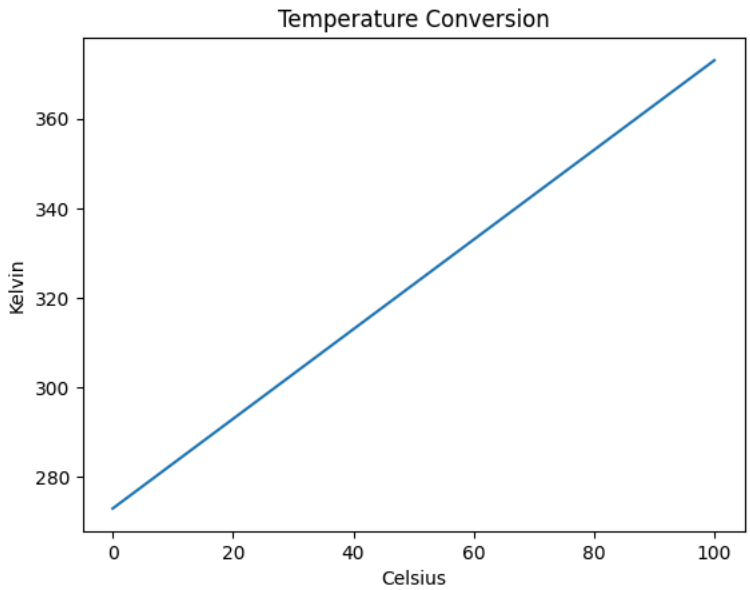
```
    __main__.BankAccount.deposit
    __main__.BankAccount.display
    __main__.BankAccount.withdraw
    __main__.Book
    __main__.Book.__init__
    __main__.Book.total_pages
    __main__.Car
    __main__.Car.__init__
    __main__.Car.display_info
    __main__.Card
    __main__.Card.__eq__
    __main__.Card.__init__
    __main__.Card.__repr__
    __main__.Chapter
    __main__.Chapter.__init__
    __main__.Circle
    __main__.Circle.__init__
    __main__.Circle.area
    __main__.CommissionEmployee
```

```
"""
Q17
Program: Scope Resolution
Author: Jasstin Roy
"""

species = "Global Species"

class Animal:
    species = "Class Species"

    def __init__(self, species):
        self.species = species

    def display_species(self):
        print("Instance species:", self.species)
        print("Class species:", Animal.species)
        print("Global species:", globals()['species'])

a = Animal("Instance Species")
a.display_species()

print("Jasstin Roy")
```

```
Instance species: Instance Species
Class species: Class Species
Global species: Global Species
Jasstin Roy
```

```
"""
Q18
Program: Celsius to Kelvin using Lambda + Pandas + Plot
Author: Jasstin Roy
"""

import pandas as pd
import matplotlib.pyplot as plt

celsius = list(range(0, 101, 10))
kelvin = list(map(lambda x: x + 273, celsius))

df = pd.DataFrame({
    "Celsius": celsius,
    "Kelvin": kelvin
})

print(df)

plt.plot(df["Celsius"], df["Kelvin"])
plt.xlabel("Celsius")
plt.ylabel("Kelvin")
plt.title("Temperature Conversion")
plt.show()

print("Jasstin Roy")
```

```
    Celsius  Kelvin
0         0     273
1        10     283
2        20     293
3        30     303
4        40     313
5        50     323
6        60     333
7        70     343
8        80     353
9        90     363
10      100     373
```




```
Jasstin Roy
```